# Self-Distilling Rewards from Environment Feedback

NeurIPS 2026

Rohan Sikand · Stanford University

2026

# Motivation

# Where Do Reward Signals Come From?

RL post-training needs a reward signal. Each existing paradigm has a fundamental failure mode:

| Paradigm | Source | Failure mode |
|---|---|---|
| Learned RM (RLHF) | Human preferences | Reward hacking (Goodhart) |
| External LLM judge | Prompted frontier model | Distributional gap $\rightarrow$ evasion |
| Self-reward | Policy judges itself | Reward collapse (no grounding) |
| Deterministic verifier (RLVR) | Binary pass/fail | Sparse: no partial credit |

We propose a fifth option: **the policy judges itself, grounded in environment feedback.**

On-policy (no gap), dense (not binary), grounded (no collapse), cheap (no external model).

# The Observation

In any RL post-training loop, the environment produces **more information than is used for reward**.

| Domain | Feedback produced | Currently used as |
|---|---|---|
| Code generation | Stack traces, compiler errors, test outputs | Binary pass/fail |
| Math reasoning | Interpreter errors, assertion failures | Binary correct/wrong |
| Tool use / agents | API errors, timeout logs, partial outputs | Binary success/fail |
| Formal verification | Proof checker messages, counterexamples | Binary valid/invalid |

Rich structured feedback is **produced and discarded**. The policy can interpret it.

A model that reads its own code alongside `IndexError at line 42` understands *why* it failed.

# Framework

# Self-Distilling Rewards: The General Mechanism

Same model $\pi_\theta$, two roles, differing only in input:

$$\underbrace{\pi_\theta(\cdot \mid s)}_{\text{Student: generates output}} \qquad \underbrace{\pi_\theta(\cdot \mid s, c_i, p_{\text{judge}})}_{\text{Teacher: evaluates output}}$$

where $c_i = (a_i, f_i, r_i)$ = output + **environment feedback** + reasoning trace.

**Three defining properties:**

- **Parameter-aligned:** same weights, zero capability gap
- **Grounded:** conditioned on external env feedback
- **Dense:** structured judgment $\rightarrow$ continuous reward

**Domain-agnostic:**

- Judgment classes change per domain
- Feedback source changes per domain
- *Mechanism is invariant*

# Dense Reward via Structured Judgment

Teacher outputs judgment over domain-specific classes $\mathcal{K}$:

$$R_{\text{self}}(s, a_i) = \sum_{k \in \mathcal{K}} w_k \cdot P_\theta(q = k \mid s, c_i, p_{\text{judge}})$$

**Code instantiation:**

| Class | $w_k$ |
|---|---|
| `correct` | $+1.0$ |
| `minor_bug` | $+0.6$ |
| `wrong_approach` | $+0.2$ |
| `critical_error` | $0.0$ |
| `exploit` | $-0.5$ |

**Why structured classes:**

- Natural factorization of failure modes
- More stable gradients than scalar prediction
- Interpretable teacher assessment
- Exploit class for hack suppression

Result: $R_{\text{self}} \in [-0.5, 1.0]$. 90%-correct code gets $\approx 0.8$, not binary 0.

# Why Grounding is the Key Ingredient

What distinguishes this from prior self-reward:

**1. Objectivity** — environment feedback is external and unforgeable. Stack traces, compiler errors, proof checker messages are produced by the execution environment, not by the model. Unlike ground-truth solutions (OPSD/SDPO), they cannot be hallucinated.

**2. Collapse prevention** — without $f_i$, self-reward degrades to sycophancy: the model convinces itself incorrect outputs are correct. With $f_i$, `IndexError at line 42` is an inescapable constraint the teacher must incorporate.

**3. Information recovery:**

$$I(Q^*; f \mid R_{\text{env}}, a) = H(Q^* \mid R_{\text{env}}, a) - H(Q^* \mid R_{\text{env}}, a, f) > 0$$

Among failing rollouts, feedback resolves whether the output was nearly correct or total

# Label-Free Judge Maintenance

**Problem:** RL shifts $\theta$ toward generation quality. Teacher quality may degrade.

Prior work: labeled replay buffer $\rightarrow$ stale as policy evolves.

**Two self-supervised objectives** from signals already in the training loop:

**Feedback prediction** — predict $f_i$ from $(s, a_i)$:

$$\mathcal{L}_{\text{pred}}(\theta) = -\mathbb{E}\left[\frac{1}{|f_i|}\sum_k \log \pi_\theta(f_{i,k} \mid s, a_i, f_{i,<k})\right]$$

{Internalizes input $\rightarrow$ feedback causality. Always on-distribution. Domain-agnostic.}

**Verifier calibration** — anchor $R_{\text{self}}$ to ground truth:

$$\mathcal{L}_{\text{cal}}(\theta) = \mathbb{E}\Big[(R_{\text{self}}(s, a_i) - R_{\text{env}}(s, a_i))^2\Big]$$

# Multiple Benefits from One Mechanism

**Dense reward**

- Partial credit for near-correct outputs
- Non-zero gradient in all-same regimes
- Failure-mode-specific shaping

**On-policy adaptation**

- No distributional gap
- Teacher evolves with student
- No evasion possible

**Reward hacking suppression**

- Explicit exploit judgment class
- Teacher reads the execution evidence
- $\mathcal{L}_{\text{pred}}$ constrains internal model

**Compute efficiency**

- Denser signal $\rightarrow$ fewer rollouts needed
- One forward pass, no external model
- No API calls, no inference budget

These are not separate contributions — they all flow from the single mechanism of grounded self-judgment.

# Self-Judge Hacking: Attack Surface and Defense

Can the policy learn to hack its own judge?

**Three lines of defense:**

1. $\mathcal{L}_{\text{cal}}$ prevents global miscalibration — systematic leniency on failing rollouts directly increases calibration loss
2. $\mathcal{L}_{\text{pred}}$ constrains internal execution model — a model that accurately predicts `IndexError` cannot simultaneously be lenient about `IndexError` when judging (shared representations)
3. **Consistency monitoring** — track correlation between $R_{\text{self}}$ and error-severity heuristics over training

**Residual risk:** local leniency on specific failure types while aggregate stays calibrated. Measured in Experiment 4.

# Theory

# Theoretical Motivation (General Results)

These results apply to **any instantiation** of the framework, not just code:

**Proposition 1: Gradient variance reduction**

Binary: $\mathrm{Var}[\hat{A}] \propto p(1-p)/G \rightarrow$ vanishes at extreme $p$

Dense: $\mathrm{Var}[\hat{A}] \geq \sigma_R^2/G > 0$ regardless of $p$

$\rightarrow$ Dense reward from *any* grounded source eliminates gradient dead zones.

**Proposition 2: Information-theoretic bound**

$I(Q^*; f \mid R_{\mathrm{env}}, a) > 0 \rightarrow$ quantifies the value of feedback in *any* domain.

**Proposition 3: Calibration safety**

If $\mathcal{L}_{\mathrm{cal}} = 0$: optimal policies under $R_{\mathrm{self}} \subseteq$ optimal policies under $R_{\mathrm{env}}$

# Experiments

# Setup

**Framework:** verl + FSDP2 + vLLM

**Models:** Qwen3-4B (primary) · Qwen3-14B (scale) · OLMo3-7B (secondary)

**Two feedback modalities** demonstrating framework generality:

- **Code:** LeetCode, LiveCodeBench — stack traces, compiler errors
- **Math:** MATH, AIME — interpreter errors, assertion failures

| Method | Dense? | Adapts? | Labels? | Reward |
|---|---|---|---|---|
| GRPO (binary) | ✗ | N/A | No | $R_{\text{env}} \in \{0, 1\}$ |
| Test-case fraction | ✓ | N/A | No | $R = k/n$ |
| External judge (Haiku) | ✓ | Static | No | LLM score |
| Ungrounded self-reward | ✓ | On-policy | No | No $f_i$ |
| EGSR + $\mathcal{L}_{\text{CE}}$ | ✓ | On-policy | Yes | Labeled buffer |
| **EGSR (label-free)** | ✓ | On-policy | **No** | $\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$ |

# Signal Validation (E1)

Frozen Qwen3-4B, 5K rollouts, no RL. Teacher AUC under context ablations, **in both domains**:

| | Code | | Math | |
|---|---|---|---|---|
| **Context** | **AUC** | **Cal.** | **AUC** | **Cal.** |
| (a) Solution only | **[X.XX]** | **[X.XX]** | **[X.XX]** | **[X.XX]** |
| (b) + env feedback | **[X.XX]** | **[X.XX]** | **[X.XX]** | **[X.XX]** |
| (c) + reasoning trace | **[X.XX]** | **[X.XX]** | **[X.XX]** | **[X.XX]** |
| (d) Full context | **[X.XX]** | **[X.XX]** | **[X.XX]** | **[X.XX]** |

**[Key: does grounding help in both modalities? Validates framework generality.]**

MI estimate: $I(Q^*; f \mid R_{\text{env}}{=}0, a) =$ **[X.XX]** bits (code), **[X.XX]** bits (math)

# Main Results (E2)

| Method | LeetCode | LiveCodeBench | MATH | AIME |
|---|---|---|---|---|
| GRPO (binary) | [XX.X] | [XX.X] | [XX.X] | [XX.X] |
| Test-case fraction | [XX.X] | [XX.X] | [XX.X] | [XX.X] |
| External judge (Haiku) | [XX.X] | [XX.X] | [XX.X] | [XX.X] |
| Ungrounded self-reward | [XX.X] | [XX.X] | [XX.X] | [XX.X] |
| EGSR + $\mathcal{L}_{\text{CE}}$ | [XX.X] | [XX.X] | [XX.X] | [XX.X] |
| **EGSR (label-free)** | [XX.X] | [XX.X] | [XX.X] | [XX.X] |

**[Key: consistent improvement across both domains demonstrates framework, not a domain-specific trick.]**

# Convergence and Gradient Analysis

**Convergence curves**

**[Figure: Pass@1 vs training steps for all methods. EGSR converges faster and higher.]**

**[Steps to target: EGSR reaches X% at step Y vs step Z for binary GRPO.]**

**Gradient norm by pass rate**

**[Figure: Gradient norm stratified by p ($<$0.2, 0.2–0.8, $>$0.8). Binary collapses at extremes; EGSR maintains signal.]**

Validates Proposition 1.

**Within-bucket analysis:** Among rollouts with identical $k/n$, EGSR assigns different $R_{\text{self}}$ that better predicts **[edit distance / full-suite pass rate]**. Self-judgment adds *qualitatively different* signal.

# Stratified Analysis

| Method | Easy | Medium | Hard |
|---|---|---|---|
| GRPO (binary) | **[XX.X]** | **[XX.X]** | **[XX.X]** |
| EGSR | **[XX.X]** | **[XX.X]** | **[XX.X]** |
| $\Delta$ | **[+X.X]** | **[+X.X]** | **[+X.X]** |

**[Expected: largest gains on medium difficulty — diverse failure modes, mixed pass rates, rich feedback signal.]**

This tells us *when* the framework helps most: problems at the learning frontier where the environment produces informative feedback about varied failure modes.

# Judge Maintenance (E3) — Key Novel Result

Teacher AUC tracked every 20 steps throughout RL training:

**[Figure: AUC vs training step for 5 conditions. (a) no maintenance — degrades; (b) $\mathcal{L}_{\text{pred}}$ only; (c) $\mathcal{L}_{\text{cal}}$ only; (d) $\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$ — stays flat; (e) labeled buffer — degrades slowly.]**

**Key findings:**

- **[(a) degrades by X AUC]**
- **[(e) degrades by Y (stale buffer)]**
- **[(d) maintains within Z AUC]**

**General implication:**

- Self-supervised maintenance works
- Always on-distribution $>$ static buffer
- Applicable to any feedback modality

# Reward Hacking and Scale (E4–E5)

**E4: Hacking dynamics**

**[Figure: Hack rate vs training step. Binary GRPO / EGSR w/o exploit / EGSR w/ exploit.]**

**[Grounded self-judgment as a reward hacking detector — a second application of the framework beyond dense reward.]**

**E5: Scale and transfer**

| | 4B | 14B |
|---|---|---|
| $\Delta$ Pass@1 | **[+X.X]** | **[+X.X]** |
| Teacher AUC | **[X.XX]** | **[X.XX]** |

**[Two domains confirm framework transfers across feedback modalities.]**

# Ablations

**Judgment granularity**

| | Pass@1 |
|---|---|
| Binary | **[XX.X]** |
| 5-class | **[XX.X]** |

**Group size**

| | $G$=4 | $G$=8 | $G$=16 |
|---|---|---|---|
| GRPO | **[X]** | **[X]** | **[X]** |
| EGSR | **[X]** | **[X]** | **[X]** |

**[EGSR $G$=4 $\approx$ GRPO $G$=16? $\rightarrow$ 4$\times$ compute savings]**

**$\lambda$ sensitivity**

**[Heatmap of $\lambda_1 \times \lambda_2$. Stable across order of magnitude?]**

**Warm-start SFT**

| | Pass@1 |
|---|---|
| No warm-start | **[XX.X]** |
| With warm-start | **[XX.X]** |

**[Can $\mathcal{L}_{\text{pred}}$ bootstrap from scratch?]**

# Discussion

# The Framework Beyond RLVR

Self-distilling rewards from environment feedback applies wherever there is structured post-hoc signal:

| **Setting** | **Grounding source** | **Transfer channel** | **Status** |
|---|---|---|---|
| RLVR (code) | Execution feedback | Reward shaping | *This paper* |
| RLVR (math) | Interpreter errors | Reward shaping | *This paper* |
| RLHF | Human pref + rationale | Reward densification | Future |
| LLM-as-judge | External judge critique | Distill then anneal | Future |
| Agents / tools | API errors, logs | Shaped plan reward | Future |
| Formal verification | Proof checker messages | Partial credit | Future |

The core mechanism transfers: same model, privileged post-hoc context, grounded judgment. The grounding source changes per domain; the framework doesn't.

# Limitations and Future Work

**Limitations:**

- Framework value scales with **feedback richness** — weaker in minimal-feedback domains
- Self-judgment capacity at small scales (4B) may be insufficient **[report]**
- Local leniency attack surface cannot be fully eliminated **[report]**
- Validated on two domains; broader validation needed

**Future directions:**

- **Non-verifiable domains:** NLI for summarization, format checkers for instruction following — weaker grounding, still constraining
- **Step-level self-reward:** extend teacher to judge individual reasoning steps (self-supervised PRM from partial execution traces)
- **Curriculum from self-judgment:** variable $R_{\text{self}}$ identifies the learning frontier

# Summary

**Framework:** Self-distilling rewards from environment feedback — a general mechanism where the policy grounds its self-judgment in structured environment output.

**Key results (RLVR instantiation):**

- **[X%]** improvement over binary GRPO across code and math
- Matches external LLM judge at **[Y×]** lower cost
- Label-free maintenance outperforms static labeled buffer
- **[Z×]** compute reduction via smaller group size

**Contributions:**

1. General framework for on-policy, grounded self-reward from environment feedback
2. Label-free judge maintenance via $\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$
3. Theoretical guarantees: variance reduction, information bound, calibration safety
4. Multiple benefits from one mechanism: dense reward, hack suppression, compute